

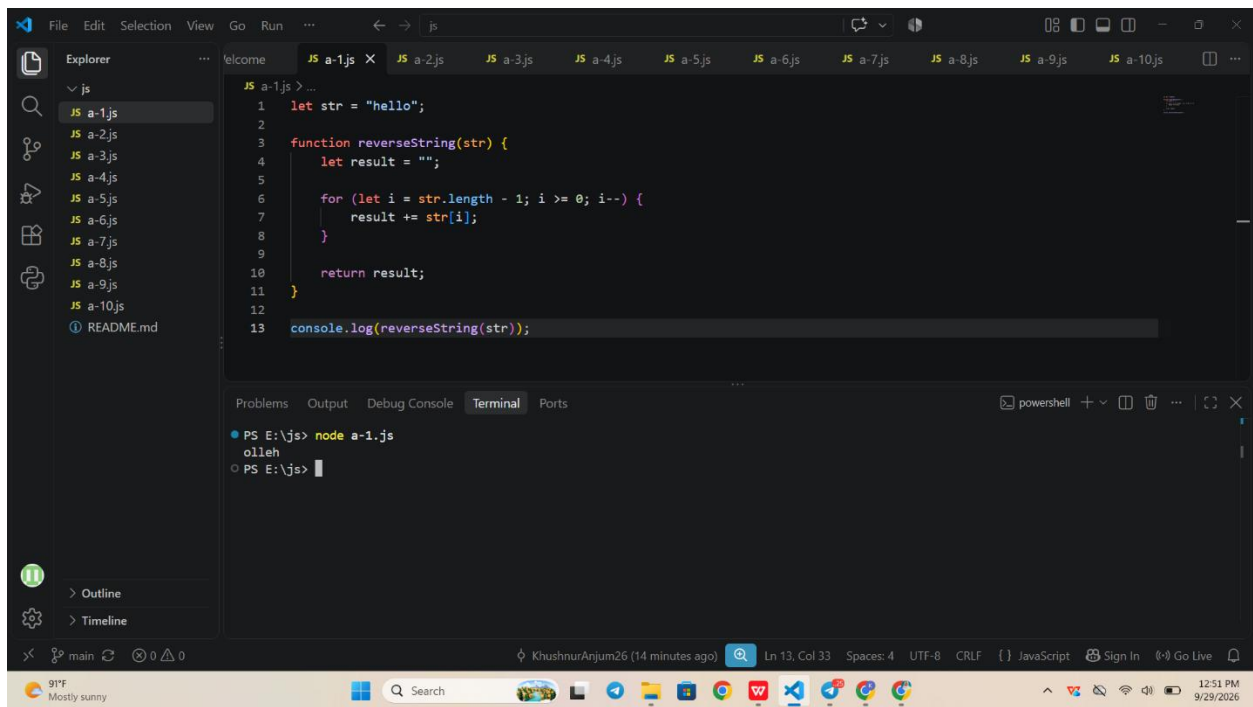


Fig-01: Reverse String

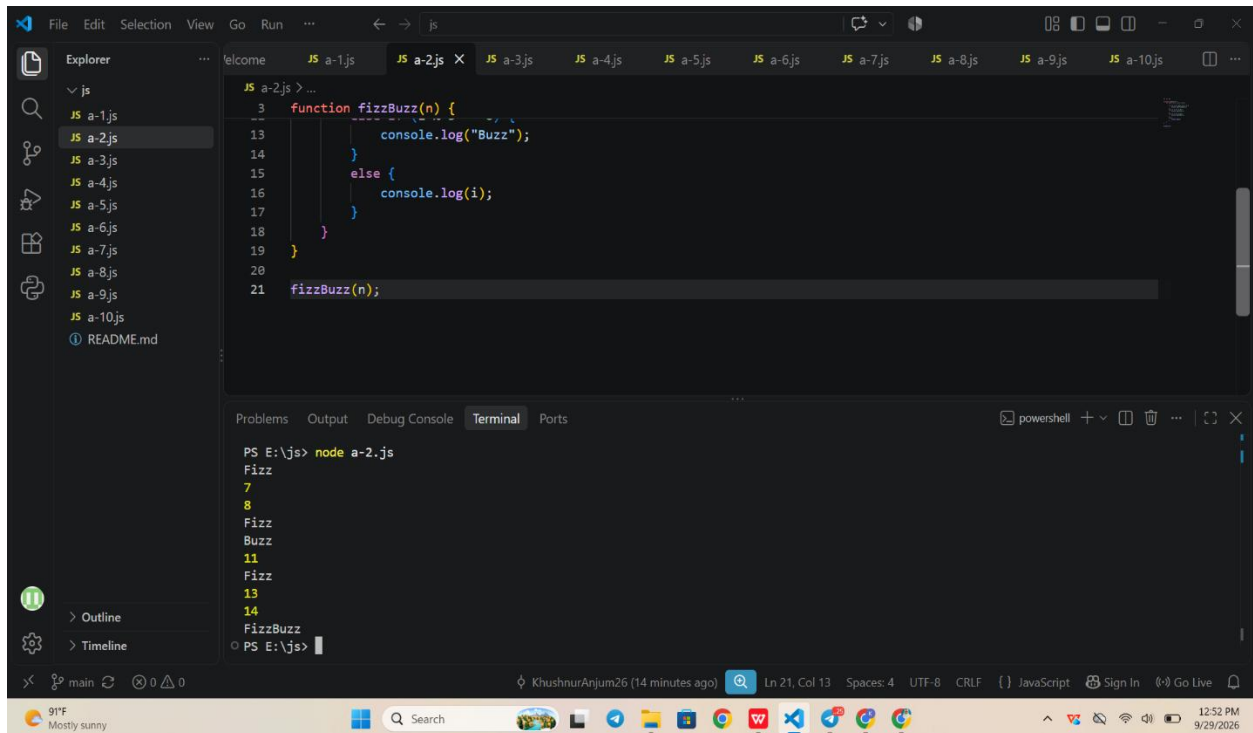


Fig-02: FizzBuzz

```
JS a-3.js > ...
1 let arr = [1, 5, 8, 3];
2
3 function findMax(arr) {
4   let max = arr[0];
5
6   for (let i = 1; i < arr.length; i++) {
7     if (arr[i] > max) {
8       max = arr[i];
9     }
10  }
11
12  return max;
13 }
14
15 console.log(findMax(arr));
```

```
PS E:\js> node a-3.js
8
PS E:\js>
```

Fig-03: Find the Largest

Number

```
JS a-4.js > countVowels
1 let str = "javascript";
2
3 function countVowels(str) {
4   let count = 0;
5
6   for (let i = 0; i < str.length; i++) {
7
8     if (str[i] == "a" || str[i] == "e" || str[i] == "i" || str[i] == "o" || str[i] == "u") {
9
10      count++;
11    }
12  }
13
14  return count;
15 }
```

```
PS E:\js> node a-4.js
3
PS E:\js>
```

Fig-04: Count Vowels

```
JS a-5.js > ...
1 let arr = [1, 2, 2, 3, 4, 4];
2
3 function removeDuplicates(arr) {
4   let result = [];
5
6   for (let i = 0; i < arr.length; i++) {
7
8     if (!result.includes(arr[i])) {
9       result.push(arr[i]);
10    }
11  }
12
13  return result;
14 }
15
```

```
PS E:\js> node a-5.js
[ 1, 2, 3, 4 ]
PS E:\js>
```

Fig-05: Remove Duplicates from Array

```
JS a-6.js > ...
1 let str = "racecar";
2
3 function isPalindrome(str) {
4
5   str = str.toLowerCase();
6
7   let reverse = "";
8
9   for (let i = str.length - 1; i >= 0; i--) {
10     reverse += str[i];
11   }
12
13   if (str == reverse) {
14     return true;
15   }
16 }
17
```

```
PS E:\js> node a-6.js
true
PS E:\js>
```

Fig-06: Check for Palindrome

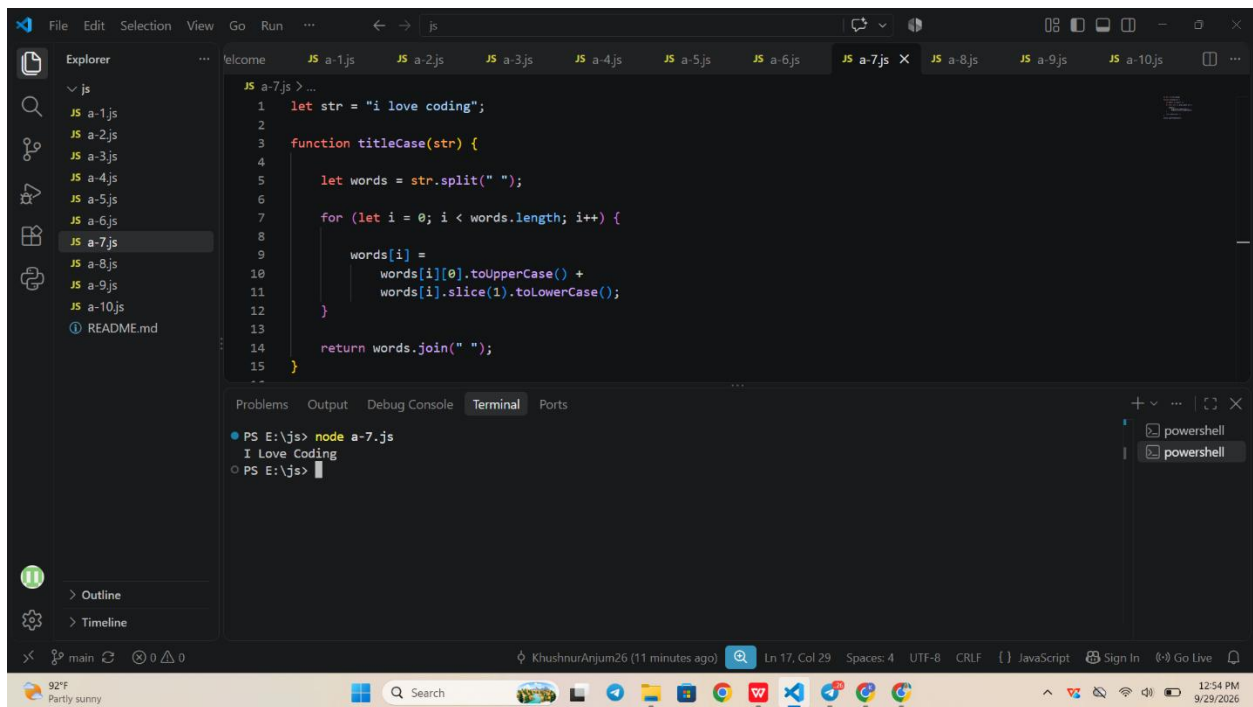


Fig-07: Title Case a Sentence

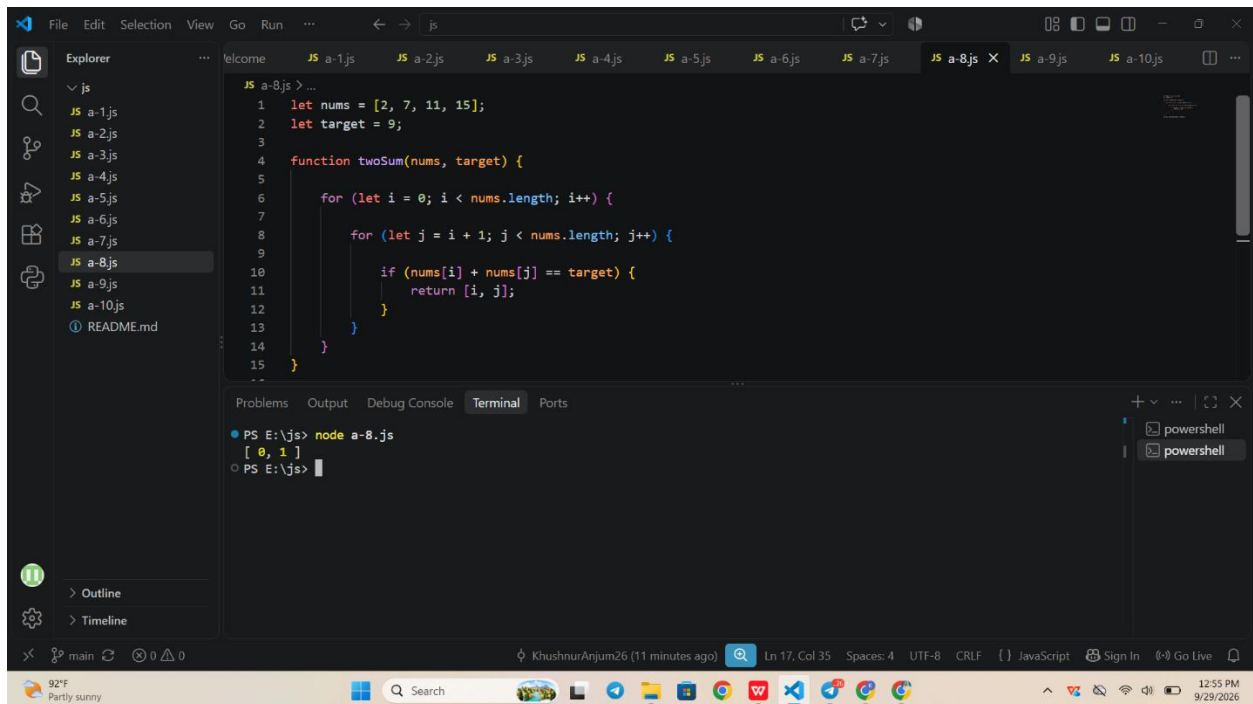


Fig-08: Two Sum Algorithm

The screenshot shows the Visual Studio Code editor with a file explorer on the left containing files JS a-1.js through JS a-10.js and a README.md file. The main editor displays the code for JS a-9.js, which defines a `memoize(fn)` function. This function creates a `cache` object and returns a wrapper function `function(n)`. The wrapper checks if `cache[n]` exists; if so, it logs "From cache" and returns the cached value. Otherwise, it calls `fn(n)`, caches the result, and returns it. The terminal at the bottom shows the command `node a-9.js` being executed, resulting in the output `120`, `From cache`, and `120`.

```
JS a-9.js > ...
1  function memoize(fn) {
2
3      let cache = {};
4
5      return function(n) {
6
7          if (cache[n]) {
8              console.log("From cache");
9              return cache[n];
10         }
11
12         let result = fn(n);
13
14         cache[n] = result;
15     }
16 }
```

```
PS E:\js> node a-9.js
120
From cache
120
PS E:\js>
```

Fig-09: Memoized Function Decorator

The screenshot shows the Visual Studio Code editor with the file explorer on the left. The main editor displays the code for JS a-10.js, which defines a `fetchWithTimeout(url, ms)` function. This function uses `fetch(url)` to make a request and a `Promise` with `setTimeout` to create a timeout. It then uses `Promise.race` to wait for either the request to complete or the timeout to occur. If the request completes first, it returns the response; otherwise, it rejects with "Request Timed Out". The terminal shows the command `node a-10.js` being executed, resulting in the output `Request Timed Out`.

```
JS a-10.js > ...
1  function fetchWithTimeout(url, ms) {
2
3      let request = fetch(url);
4
5      let timeout = new Promise((resolve, reject) => {
6
7          setTimeout(() => {
8              reject("Request Timed Out");
9          }, ms);
10     });
11
12     return Promise.race([request, timeout]);
13 }
14
15
16 fetchWithTimeout("https://jsonplaceholder.typicode.com/posts", 90)
17   .then(response => response.json())
18   .then(data => console.log(data))
19   .catch(error => console.log(error));
```

```
PS E:\js> node a-10.js
Request Timed Out
PS E:\js>
```

Fig-10: Asynchronous Fetch Timeout Wrapper